



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
DEPARTAMENTO DE INGENIERÍA INDUSTRIAL Y DE SISTEMAS
ICS2121 - MÉTODOS DE OPTIMIZACIÓN - 2025-2

Profesor: JORGE VERA

Ayudantes: - CHRISTOPH BÜRGER (CHRISTOPH.BUERGER@UC.CL)
- FRANCISCO TAMARÍN (FTAMARIN@UC.CL)
- ALBERTO URETA (ALBERTO.URETA@UC.CL)

Taller 5

Repaso I1

4 de septiembre de 2025

Resumen

- **Condición de Lipschitz:** $|f(x) - f(y)| \leq L |x - y| \quad \forall x, y$
- **Método de Gradiente (Convergencia lineal):**
 1. Sea $x^0 \in \mathbb{R}^n, k = 0, \epsilon > 0$.
 2. Sea $d^k = -\nabla f(x^k)$
 3. Si $\|\nabla f(x^k)\| \leq \epsilon$, entonces STOP.
 4. Sea λ^k solución de $\min_{\lambda \geq 0} f(x^k + \lambda d^k)$
 5. Sea $x^{k+1} = x^k + \lambda^k d^k, k \leftarrow k + 1$, go to 2.
- **Método de Newton (Convergencia cuadrática):**
 1. Sea $x^0 \in \mathbb{R}^n, k = 0, \epsilon > 0$.
 2. Sea $d^k = -[\nabla^2 f(x^k)]^{-1} \nabla f(x^k)$
 3. Si $\|\nabla f(x^k)\| \leq \epsilon$, entonces STOP.
 4. Sea λ^k solución de $\min_{\lambda \geq 0} f(x^k + \lambda d^k)$
 5. Sea $x^{k+1} = x^k + \lambda^k d^k, k \leftarrow k + 1$, go to 2.
- **Método simple de primer orden:**
 1. Sea $x^0 \in \mathbb{R}^n, k = 0, d^0 = -\nabla f(x^0), \epsilon > 0$.
 2. Sea $d^k = -\nabla f(x^k)$
 3. Si $\|\nabla f(x^k)\| \leq \epsilon$, entonces STOP.
 4. Sea $x^{k+1} = x^k + \frac{1}{L} d^k, k \leftarrow k + 1$, go to 2.

■ **Método del subgradiente:**

1. Sea $x^0 \in \mathbb{R}^n, k = 0, -d^0 \in \partial f(x^0), \epsilon > 0$.
2. Sea $-d^k \in \partial f(x^k)$
3. Si el criterio de parada se satisface, entonces STOP. En caso contrario, continuar con el paso siguiente.
4. Sea $x^{k+1} = x^k + \lambda_k d^k, k \leftarrow k + 1$, go to 2.

■ **Gradiente**

El Gradiente de una función $f : \mathbb{R}^n \rightarrow \mathbb{R}$ se define como sigue:

$$\nabla f(x) = \begin{pmatrix} \frac{\partial f(x)}{\partial x_1} \\ \frac{\partial f(x)}{\partial x_2} \\ \vdots \\ \frac{\partial f(x)}{\partial x_n} \end{pmatrix}$$

■ **Matriz Hessiana**

Sea una función $f : \mathbb{R}^n \rightarrow \mathbb{R}$. Se define la matriz hessiana como sigue:

$$\nabla^2 f(x) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$

- **Dirección de descenso:** d es dirección de descenso si cumple que $d^\top \nabla f(x) < 0$

Ejercicio 1

- (a) Supongamos que queremos resolver el problema $\min_{x \in \mathbb{R}^n} f(x)$, donde f es una función fuertemente convexa y tal que ∇f cumple una propiedad de Lipschitz. Nos proponen usar el Algoritmo de Primer Orden Acelerado de Nesterov o el Método Simple de Primer Orden (con paso fijo). En términos del orden de número iteraciones necesario para llegar a una solución ϵ -óptima, cuál de los dos algoritmos tiene mejor complejidad? ¿Cuál es esa complejidad? Justifique. No use más de 4-5 líneas para responder esta pregunta.

Solución: Poseen la misma complejidad, por la propiedad de convexidad fuerte, siendo en este caso $\mathcal{O}(\log(\frac{1}{\epsilon}))$

- (b) Indique, en forma justificada, sobre la base de los contenidos del curso, si la siguiente afirmación es correcta o no: “Para el problema no restringido $\min_{x \in \mathbb{R}^n} f(x)$, el método de Newton converge siempre a velocidad cuadrática, para cualquier tipo de función dos veces continuamente diferenciable, siempre y cuando el punto de partida esté suficientemente cerca de una solución óptima”. No use más de 4-5 líneas para responder esta pregunta.

Solución: La afirmación es Falsa, dado que se necesita que el Hessiano sea definido positivo además.

- (c) Considere el problema

$$\begin{array}{ll} \min & f(x) \\ \text{s. a.} & 0 \leq x_j \leq a_j, j = 1, \dots, n \end{array}$$

donde los a_j son escalares positivos y $f : \mathbb{R}^n \rightarrow \mathbb{R}$ es una función dos veces continuamente diferenciable y convexa, pero no necesariamente se cumple que ∇f tenga una propiedad de Lipschitz en todo el espacio $\mathbb{R}^n$. Argumente que, sin embargo, sí es posible asumir la existencia de una constante de Lipschitz “local” para ∇f en el conjunto definido por las restricciones del problema. Explique cómo podría calcular esa constante de Lipschitz si le dieran una función f explícitamente.

Solución: Sea S el conjunto definido por las restricciones del problema. Si $f(x)$ cumple que es acotada en S , entonces se puede estimar L válido como $L = \max_{x \in S} \|\nabla^2 f(x)\|$.

- (d) Considere la función de n variables dada por:

$$f(x) = \frac{1}{2}x^\top Qx + \sum_{j=1}^n e^{-x_j^2}$$

donde $Q \in \mathbb{R}^{n \times n}$ es simétrica y definida positiva.. ¿Cumple $\nabla f(x)$ una propiedad de Lipschitz en todo el espacio $\mathbb{R}^n$? Si su respuesta es afirmativa, determine una estimación

al valor de esa constante y si su respuesta es negativa justifique por qué no. Sea claro en todas sus explicaciones y desarrollo matemático, y supuestos que haga.

Solución: El Hessiano corresponde a

$$\nabla^2 f(x) = Q + \begin{bmatrix} e^{-x_1^2} \cdot (4x_1^2 - 2) & 0 & \cdots & 0 \\ 0 & e^{-x_2^2} \cdot (4x_2^2 - 2) & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & e^{-x_n^2} \cdot (4x_n^2 - 2) \end{bmatrix}$$

Notar que es acotado. Así se puede determinar L tal que $\|\nabla^2 f(x)\| \leq L$.

Ejercicio 2

Consideremos el siguiente problema de optimización, que surge en algunas variantes de problemas de estimación estadística. Queremos resolver un problema de estimar un vector $b \in \mathbb{R}^m$ a partir de observaciones de otros n factores, las cuales están tabuladas en una matriz $A \in \mathbb{R}^{m \times n}$, pero donde n es mayor que m . Entonces, queremos encontrar una solución al sistema $Ax = b$. Ahora, en el curso hemos tratado de imponer una condición de que x tenga el menor número de elementos no nulos. Acá haremos algo un poco diferente: queremos minimizar el “largo” del vector x , que se define por $l(x) = \min\{k : x_j = 0, j > k\}$. Esto corresponde al menor índice de coordenadas a partir de la cual todas las otras son nulas. Es como buscar un vector x con la menor cantidad de coordenadas no nulas, pero que estén al comienzo del vector. Por ejemplo, con $n = 10$:

$$\begin{aligned}x &= (1, 2, 4, 0, 0, 0, 0, 0, 0, 0), l(x) = 3, \\y &= (1, 0, 0, 0, 0, 0, 0, 0, 8, 0), l(y) = 9.\end{aligned}$$

Podemos ver que y , pese a que tiene menos coordenadas no nulas, tiene largo 9 ya que $y_9 \neq 0$. Entonces, x es preferible a y pese a que tiene más coordenadas no nulas. Entonces, queremos resolver el problema:

$$\begin{aligned}\min \quad & l(x) \\ \text{s. a.} \quad & Ax = b\end{aligned}$$

Inspirados en lo que hicimos para el modelo LASSO, postulamos el siguiente problema aproximado, con la intención de obtener alguna solución que vaya en la dirección de nuestro objetivo:

$$\min_{x \in \mathbb{R}^n} \tau \sum_{j=1}^n \beta_j |x_j| + \|Ax - b\|_2^2$$

donde los números β_j son todos positivos y tales que $\beta_{j+1} > \beta_j$, y $\tau > 0$ es un parámetro de balance. Específicamente, tomemos la siguiente definición para estos números: $\beta_j = 2^j, j = 1, \dots, n$ (es decir, $\beta_1 = 2, \beta_2 = 4$, etc.).

- (a) Argumente que el resolver ese problema puede, efectivamente, generar una solución x que pudiera aproximarse a la estructura que buscamos, es decir, componentes x_j tales que $|x_j|$ sea bastante pequeño cuando j sea grande (y pueda ser redondeado a 0, como se hizo en clases).

Solución: Al analizar el comportamiento de la función objetivo, notamos que cada componente $|x_j|$ está penalizada con un peso $\beta_j = 2^j$, el cual crece exponencialmente con j . Esto implica que, para minimizar la función objetivo, se vuelve cada vez más costoso mantener valores distintos de cero en las posiciones con índices altos. Como resultado, el modelo tenderá a asignar valores más pequeños a los x_j con índices grandes, mientras que permitirá valores más grandes en las primeras coordenadas.

Esta estructura incentiva soluciones donde los valores no nulos de x se concentren al comienzo del vector, lo que está alineado con el objetivo de minimizar el largo $l(x)$. Por lo tanto, esta formulación aproximada puede efectivamente conducir a soluciones con la estructura deseada.

- (b) El problema planteado aquí es no diferenciable. Determine una fórmula general para $\partial f(x)$, donde f es la función objetivo del problema. Justifique con precisión todo lo que haga.

Solución: Sea $f(x) = \tau \sum_{j=1}^n \beta_j |x_j| + \|Ax - b\|_2^2$

Luego, obtenemos $\partial f(x)$ de la función:

$$\partial f(x) = \partial \left(\tau \sum_{j=1}^n \beta_j |x_j| + \|Ax - b\|_2^2 \right) = \left(\tau \cdot \partial \left(\sum_{j=1}^n \beta_j |x_j| \right) + \partial (\|Ax - b\|_2^2) \right)$$

Se puede observar que la función posee una parte diferenciable y una parte no diferenciable.

Para la parte diferenciable: $\partial (\|Ax - b\|_2^2) = 2A^\top (Ax - b)$

Para la parte no diferenciable: Definimos $p(x) \in \mathbb{R}^n$ como

$$p_j(x) = \begin{cases} \beta_j & \text{si } x_j > 0 \\ -\beta_j & \text{si } x_j < 0 \\ \mu \in [-\beta_j, \beta_j] & \text{si } x_j = 0 \end{cases}$$

Finalmente, denotamos $p(x) \in \partial f(x)$ tal que:

$$\partial f(x) = \tau \cdot p(x) + 2A^\top (Ax - b)$$

Una forma alternativa es derivando utilizando la función signo:

$$\partial f(x) = \tau \cdot \beta_j \cdot \text{sgn}(x) + 2A^\top (Ax - b)$$

- (c) Supongamos que queremos resolver este problema con el algoritmo FISTA. Justifique, según los contenidos del curso, que el problema efectivamente tiene el formato de problema que puede ser resuelto con FISTA. Y en este caso, ¿cuál sería la complejidad del algoritmo, en orden de número de iteraciones para llegar a una solución ϵ -óptima? (no use más de 5-6 líneas para responder esta pregunta).

Solución: El problema puede resolverse con el algoritmo FISTA debido a que su función objetivo se puede descomponer en la suma de dos términos: uno diferenciable y convexo, $\|Ax - b\|_2^2$, que además tiene gradiente L -Lipschitz; y otro no diferenciable pero convexo, $\sum_{j=1}^n \beta_j |x_j|$, que corresponde a una penalización ponderada tipo LASSO. La complejidad del algoritmo en términos del número de iteraciones para alcanzar una solución ϵ -óptima es $\mathcal{O}(1/\sqrt{\epsilon})$.

-
- (d) Suponga que ahora se quiere resolver una variante restringida del problema, de la forma:

$$\begin{aligned} \text{mín} \quad & \tau \sum_{j=1}^n \beta_j |x_j| + \|Ax - b\|_2^2 \\ \text{s. a.} \quad & l_j \leq x_j \leq u_j, j = 1, \dots, n \end{aligned}$$

donde $l_j < u_j, j = 1, \dots, n$. Si se usa ahora una versión proyectada del algoritmo FISTA, explique cómo se resuelve la etapa de proyección que debe incorporar en el algoritmo.

Solución: Para esto, se debe proyectar x en cada iteración del algoritmo, sobre el conjunto $D = \{x : l_j \leq x_j \leq u_j \forall j\}$.

Lo que se debe hacer es revisar cada cordenada x_j y determinar si es que está se encuentra dentro del conjunto D . Si $l_j \leq x_j \leq u_j$, entonces esta coordenada queda igual. Por otro lado, si $x_j < l_j$, esta coordenada se debe poner en l_j y si $x_j > u_j$, se debe llevar a u_j . Esto es:

$$\Pi_D(x)_j = \begin{cases} x_j & \text{si } l_j \leq x_j \leq u_j \\ l_j & \text{si } x_j < l_j \\ u_j & \text{si } x_j > u_j \end{cases}$$

Se puede apoyar la respuesta con un dibujo, pero este debe ser explicado adecuadamente.

Ejercicios Propuestos

Ejercicio 1

(a) Considere la función de n variables dada por:

$$f(x) = \frac{1}{2}x^\top Qx + \sum_{j=1}^{n-1} (x_{j+1} - x_j)^2$$

donde $Q \in \mathbb{R}^{n \times n}$ es simétrica y definida positiva. ¿Cumple $\nabla f(x)$ una propiedad de Lipschitz en todo el espacio $\mathbb{R}^n$? Si su respuesta es afirmativa, construya una estimación al valor de esa constante y si su respuesta es negativa justifique por qué no. Sea claro en todas sus explicaciones y desarrollo matemático, y supuestos que haga.

Solución: Se pide demostrar

$$\|\nabla f(x) - \nabla f(y)\|_2 \leq L\|x - y\|_2$$

que es equivalente a

$$\|\nabla^2 f(x)\|_2 \leq L$$

Definiendo

$$f(x) = \underbrace{\frac{1}{2}x^\top Qx}_{h(x)} + \underbrace{\sum_{j=1}^{n-1} (x_{j+1} - x_j)^2}_{g(x)}$$

Se tiene que

$$\begin{aligned} f(x) &= h(x) + g(x) \\ \nabla f(x) &= \nabla h(x) + \nabla g(x) \end{aligned}$$

Notar que $\nabla h(x) = Qx$. Por otro lado, se deduce que

$$\frac{\partial g(x)}{\partial x_k} = \begin{cases} 2(x_k - x_{k-1}) & k = n \\ 2(x_k - x_{k-1}) - 2(x_{k+1} - x_k) & k = 2, \dots, n-1 \\ -2(x_{k+1} - x_k) & k = 1 \end{cases}$$

Se tiene además

$$\nabla^2 f(x) = \nabla^2 h(x) + \nabla^2 g(x)$$

Se tiene que $\nabla^2 h(x) = Q$ y para obtener $\nabla^2 g(x)$ se tienen las siguientes derivadas parciales

$$\begin{aligned}\frac{\partial^2 g(x)}{\partial x_{k+1} \partial x_k} &= -2 \\ \frac{\partial^2 g(x)}{\partial x_{k-1} \partial x_k} &= -2 \\ \frac{\partial^2 g(x)}{\partial x_k^2} &= \begin{cases} 2 & k \in \{1, n\} \\ 4 & k = 2, \dots, n-1 \end{cases} \\ \frac{\partial^2 g(x)}{\partial x_l \partial x_k} &= 0 \quad |l - k| > 2\end{aligned}$$

Por tanto, como el Hessiano queda en términos de constantes, se puede obtener una constante L-Lipschitz la cual corresponde a la norma del Hessiano.

- (b) Siguiendo con la misma función de la parte (a), determine el esfuerzo computacional que requiere evaluar el gradiente de f en un punto x cualquiera. Justifique con precisión.

Solución: El esfuerzo de calcular $\nabla f(x) = Qx + \nabla g(x)$ se separa en calcular el esfuerzo de Qx que es $\mathcal{O}(n^2)$ y el esfuerzo de evaluar $\nabla g(x)$ es $\mathcal{O}(n)$. Por tanto, el esfuerzo de evaluar $\nabla f(x)$ es $\mathcal{O}(n^2)$.

- (c) Considere el siguiente problema de optimización:

$$\min_{x \in \mathbb{R}^n} \left\{ \left(\sum_{j=1}^n \max\{0, x_j\} \right) + f(x) \right\}$$

donde f es la función de la parte a). ¿Puede el algoritmo FISTA aplicarse a este problema? Justifique con precisión su respuesta de acuerdo a las características del algoritmo FISTA presentadas en el curso y de otros contenidos que necesite. Sea preciso en su justificación.

Solución: Dado que se puede obtener una constante L-Lipschitz para $\nabla f(x)$ y $\sum_{j=1}^n \max\{0, x_j\}$ es subdiferenciable, se puede ocupar el algoritmo FISTA sin problemas.

- (d) Considere un problema de la forma $\min_{x \in \mathbb{R}^n} f(x)$, donde f es una función convexa dos veces continuamente diferenciable y con hipótesis suficientes para garantizar que cualquier algoritmo de los que conocemos funcionará correctamente. También asuma que se sabe que el cálculo del gradiente requiere $\mathcal{O}(n)$ flops, el cálculo del Hessiano $\mathcal{O}(n^2)$ flops y que, eventualmente, se puede usar un método de Linesearch que hace un máximo de p iteraciones usando gradientes. Explique cómo estimaría a partir de qué dimensión n será más conveniente usar el Método Simple de Primer Orden en vez del Método de Newton para llegar a una solución ϵ -aproximada, para $0 < \epsilon < 1$ dado. Sea claro en su explicación, explicitando relaciones matemáticas adecuadas y justifique según los contenidos del curso, explicando cualquier supuesto que tenga que hacer.

Solución: Se sabe que el tiempo total del Método Simple de Primer Orden y del Método de Newton son $c_1 \cdot k_1 \cdot n \cdot \frac{1}{\epsilon}$ y $c_2 \cdot k_2 \cdot n^3 \log(\log(\frac{1}{\epsilon}))$ respectivamente, con c_1 y c_2 constantes. Asumiendo $c_1 = c_2 = c$ se debe cumplir la siguiente relación

Tiempo total Método Simple de Primer Orden $\leq$ Tiempo total Método de Newton
que se traduce en

$$c_1 \cdot k_1 \cdot \frac{1}{\epsilon} \leq c \cdot k_2 \cdot n^3 \cdot \log(\log(\frac{1}{\epsilon}))$$

despejando se obtiene

$$\sqrt{\frac{k_1}{k_2} \cdot \frac{\frac{1}{\epsilon}}{\log(\log(\frac{1}{\epsilon}))}} \leq n$$

Nota: Si alguien asume convexidad fuerte para el Método Simple de Primer Orden, en vez de $\frac{1}{\epsilon}$ se tiene $\log(\frac{1}{\epsilon})$, que es también correcto para el desarrollo. Se debe explicitar.

Ejercicio 2

(a) Considere la función de n variables dada por:

$$f(x) = \tau \sum_{j=1}^n \max\{(\alpha_j + x_j^2), 0\} + \|Ax - b\|_2^2$$

donde $\tau > 0$ es dado, los α_j son escalares no nulos (de cualquier signo), $A \in \mathbb{R}^{m \times n}$ con $n > m$, y $b \in \mathbb{R}^m$. Determine el subdiferencial de f en cualquier punto x del dominio.

Solución: Se tiene

$$\begin{aligned} \partial f(x) &= \partial \left(\tau \sum_{j=1}^n \max\{(\alpha_j + x_j^2), 0\} + \|Ax - b\|_2^2 \right) \\ &= \partial \left(\tau \sum_{j=1}^n \max\{(\alpha_j + x_j^2), 0\} \right) + \partial(\|Ax - b\|_2^2) \\ &= \tau \cdot \partial \left(\sum_{j=1}^n \max\{(\alpha_j + x_j^2), 0\} \right) + 2A^\top (Ax - b) \end{aligned}$$

Acá estamos abusando notación con ∂ pero es análogo a lo hecho en clases. El segundo sumando lo conocemos de los desarrollos mostrados en clases.

Tenemos que calcular, entonces, $\partial \left(\sum_{j=1}^n \max\{(\alpha_j + x_j^2), 0\} \right)$. Sea $p(x) \in \mathbb{R}^n$, denotamos el j -ésimo elemento $p(x)_j$ como:

$$p(x)_j = \begin{cases} 2x_j & x_j < -\sqrt{-\alpha_j} \text{ y } x_j > \sqrt{-\alpha_j} \\ 0 & x_j \in]-\sqrt{-\alpha_j}, \sqrt{-\alpha_j}[\\ \mu : \mu \in [-2\sqrt{-\alpha_j}, 0] & x_j = -\sqrt{-\alpha_j} \\ \nu : \nu \in [0, 2\sqrt{-\alpha_j}] & x_j = \sqrt{-\alpha_j} \end{cases}$$

Notar que lo anterior sólo tiene sentido para el caso en que $\alpha_j < 0$. Para el caso en que $\alpha_j \geq 0$, se tiene que la función es diferenciable en todo su dominio, por lo que $p(x)_j = 2x_j$ para todo $x_j \in \mathbb{R}$. Es necesario, sin embargo, considerar ambos casos.

Entonces, si denotamos por $h(x)$ un subgradiente de $f(x)$ en x , tenemos que

$$h(x) = \tau p(x) + 2A^\top (Ax - b)$$

(b) Considere, alternativamente, la función:

$$f(x) = \tau \sum_{j=1}^n (\alpha_j + x_j^2) + \|Ax - b\|_2^2$$

donde $\tau > 0$ es dado, los α_j son escalares no nulos, $A \in \mathbb{R}^{m \times n}$, $n > m$, y $b \in \mathbb{R}^m$. Determine si $\nabla f(x)$ cumple una propiedad de Lipschitz en todo $\mathbb{R}^n$ y estime la constante o justifique si no la cumple.

Solución: Notar que la función es cuadrática y diferenciable en todo su dominio. Obtenemos su gradiente:

$$\nabla f(x) = 2\tau x + 2A^\top(Ax - b)$$

Para que $\nabla f(x)$ cumpla con la condición de Lipschitz, debe existir un $L > 0$ constante, tal que:

$$\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\| \quad \forall x, y \in \mathbb{R}^n$$

En palabras, la variación de los gradientes está acotada por la variación de los puntos evaluados. Dado que $\nabla f(x)$ es una función lineal, su derivada no quedará en términos de x y su norma será constante. Por lo tanto, es posible acotar globalmente la norma del Hessiano y estimar L a partir de esa norma, de la misma forma que se hizo en clases para el problemas cuadrático más general. Luego, se tiene que:

$$\nabla^2 f(x) = 2\tau I + 2A^\top A$$

Con $I \in \mathbb{R}^{n \times n}$ matriz de identidad. Calculando su norma se obtiene que:

$$\|\nabla^2 f(x)\| = \|2\tau I + 2A^\top A\|$$

Dado que las segundas derivadas parciales cruzadas son todas cero (debido a que los términos cruzados son constantes), la matriz hessiana es diagonal y cada elemento en la diagonal es 2τ más la suma de los cuadrados de los elementos correspondientes de la i -ésima fila de la matriz A . De esto se obtiene que:

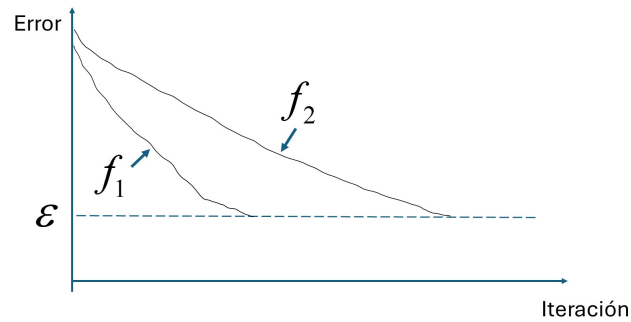
$$L = 2\tau + 2\|A^\top A\|$$

Como $A^\top A$ es semidefinida positiva, usamos la norma-2 de matrices (tal como se hizo en clases), y luego se tiene que $\|A^\top A\|_2 = \lambda_{\max}(A^\top A)$, con $\lambda_{\max}(A^\top A) \geq 0$ el máximo valor propio. Finalmente se obtiene que:

$$L = 2\tau + 2\lambda_{\max}(A^\top A)$$

Notar que $L > 0$ siempre.

- (c) El siguiente gráfico muestra, de forma conjunta, cómo evolucionan las iteraciones del Método Simple de Primer Orden aplicado a dos funciones, f_1 y f_2 , para buscar el mínimo de estas, hasta llegar a un error $\epsilon > 0$. Ambas funciones son de n variables, son convexas, sus gradientes cumplen condición de Lipschitz y no necesariamente son fuertemente convexas:



Si en ambos casos, el algoritmo parte de un mismo punto inicial y las correspondientes soluciones óptimas son cercanas, ¿para cuál de las dos funciones la constante de Lipschitz L debería ser mayor? Justifique con precisión su respuesta sobre la base de los contenidos del curso y en no más de 4 líneas.

Solución: Como se mencionó en clases, la expresión para encontrar la cantidad de iteraciones que se deben realizar para tener una solución con error $\epsilon > 0$ es:

$$\frac{L\|x^0 - x^*\|_2^2}{2\epsilon} \leq k$$

Por enunciado sabemos que el punto inicial es el mismo en ambos casos, y las soluciones óptimas no son muy diferentes, así que los términos de error son muy parecidos, por lo que el único término que diferencia significativamente a ambas funciones, respecto a la aplicación del método, es L . Así, un L mayor significará un mayor número de iteraciones para llegar a una solución con error ϵ , la cual corresponde a f_2 .

Ejercicio 3

Queremos resolver el problema $\min_{x \in \mathbb{R}^n} f(x)$, donde f es continuamente diferenciable y convexa, y tenemos que n es muy grande. Como se comentó en clases, en esos casos no se puede calcular explícitamente el vector $\nabla f(x)$ así que se recurre a algunas aproximaciones. Consideremos el algoritmo descrito por el siguiente pseudo-código, y que llamaremos “Algoritmo Misterioso”:

Algoritmo Misterioso (para minimizar):

Data: $x^0 \in \mathbb{R}^n$, f , $k = 0$, $LIMITE$, un escalar > 0 .

while $k < LIMITE$ **do**

Seleccionar $j_0 \in \{1, \dots, n\}$ en forma aleatoria. Si j_0 es igual al de la selección anterior, repetir.

Sea $d_{j_0}^k = -\frac{\partial f}{\partial x_{j_0}}(x^k)$ Sea $d_j^k = 0, j = 1, \dots, n, j \neq j_0$

Resolver aproximadamente

$$\min_{\lambda \geq 0} f(x^k + \lambda d^k) \quad (\text{etapa de “linesearch”})$$

Sea λ^k solución de ese problema

Sea $x^{k+1} = x^k + \lambda^k d^k$

$k = k + 1$

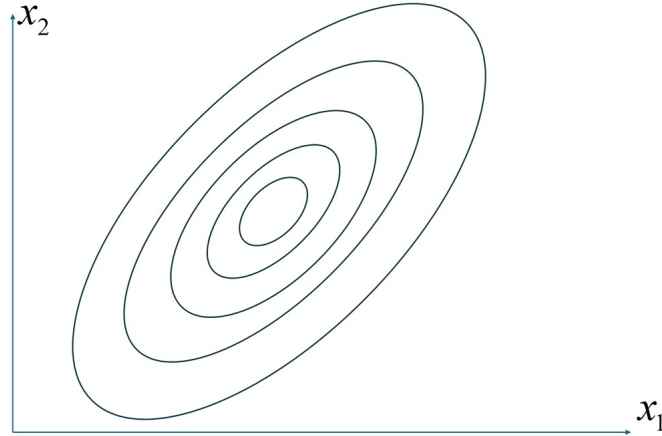
end

Respecto a la etapa 2 del método, hay una probabilidad de que se repita el índice seleccionado pero, dado que estamos diciendo que n es muy grande, realmente la probabilidad de eso es muy baja y no hay problemas si asumimos que no se repite.

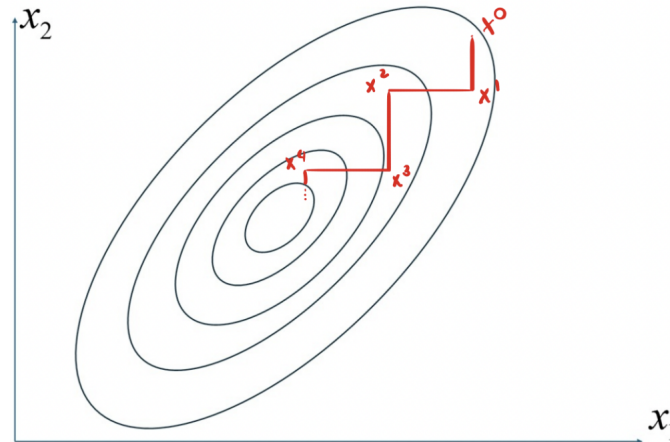
- (a) ¿Es d^k , calculada por el algoritmo, una dirección de descenso? Justifique con precisión.

Solución: Se tiene que d^k es dirección de descenso. En efecto, basta verificar $\nabla f(x^k)^\top d^k = -\left(\frac{\partial f(x^k)}{\partial x_{j_0}}\right)^2 < 0$.

- (b) Explique qué es lo que hace este “Algoritmo Misterioso”. Haga un dibujo simple (en dos variables, sí, pese a que n es grande, pero eso lo que se puede dibujar) mostrando las curvas de nivel de alguna función (cuadrática si quiere), mostrando cómo sería la trayectoria del algoritmo. Justifique con precisión su respuesta y no se extienda en más de 5-6 líneas (sin incluir el dibujo). Puede usar, como referencia, un dibujo de curvas de nivel como el siguiente:

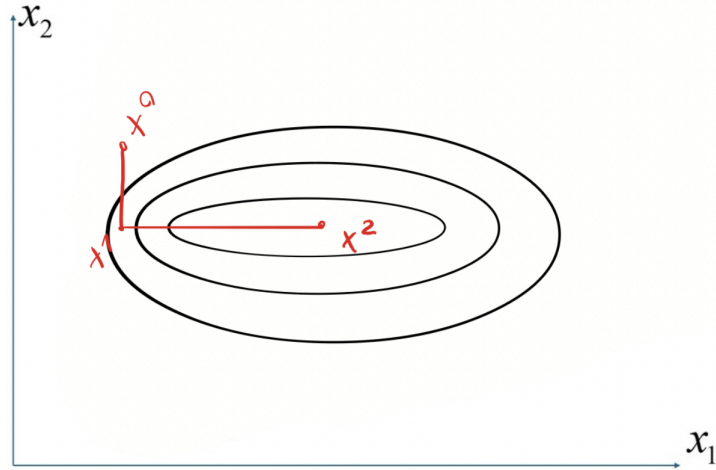


Solución: El algoritmo toma aleatoriamente direcciones de descenso paralelas a algún eje, dado que estas son siempre no nulas en sólo una coordenada. Luego, busca el mínimo mediante un Linesearch en esa dirección, y repite el proceso para otra dirección. Una posibilidad de dibujo es la siguiente:



- (c) Para este “**Algoritmo Misterioso**”, ¿hay alguna forma especial de la función objetivo que favorezca en forma significativa la convergencia? Puede apoyarse en un dibujo, pero igual debe dar un argumento basado en los contenidos del curso, y explicar con claridad.

Solución: Opción 1: Una función cuadrática de la forma $f(x) = x^\top Qx + b^\top x$ con Q diagonal, dado que $f(x) = \sum_{i=1}^N [\lambda_i x_i^2 + b_i x_i]$ se puede minimizar por separado ($\{\lambda_i\}_{i=1}^N$ son los valores propios de Q y entradas de la diagonal). Es decir, el algoritmo entrega el óptimo en a lo más N iteraciones. Gráficamente esta función se ve de la siguiente forma:



Notar que el condicionamiento no influye en la cantidad de iteraciones del algoritmo para este tipo de funciones.

Opción: 2: Es posible argumentar que bajo un mal condicionamiento de la función, el método empeora. El estudiante puede mostrar esto mediante una comparación entre dibujos, recordando que el condicionamiento de una función está directamente relacionado con la excentricidad de las curvas de nivel:

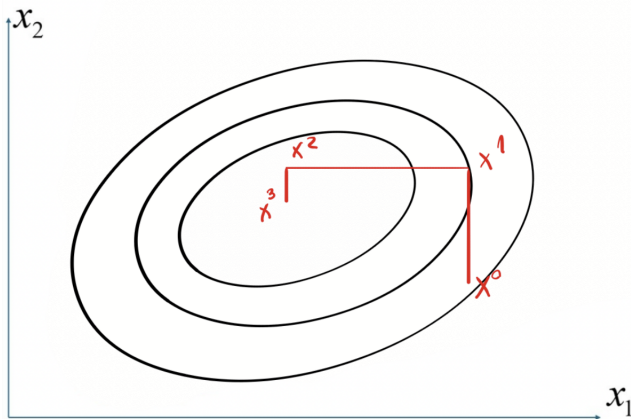


Figura 1: Algoritmo Misterioso bien condicionado

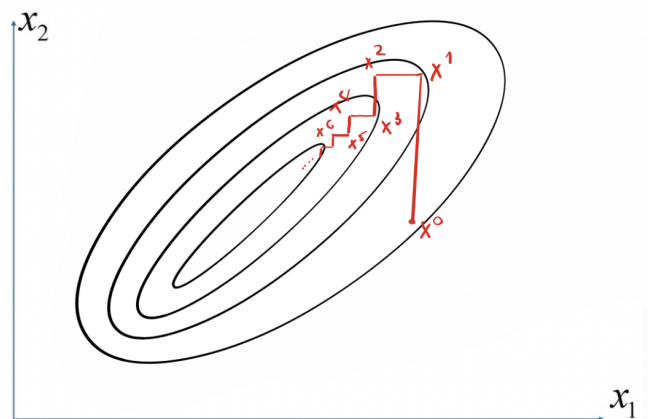


Figura 2: Algoritmo Misterioso mal condicionado

- (d) ¿Qué tan buena cree usted que será la velocidad de convergencia de este algoritmo, comparada con la de los métodos clásicos que hemos estudiado, como el método de Gradiente o el Método de Newton? Justifique su discusión en función de los contenidos del curso (Esta pregunta puede admitir diversos argumentos, pero queremos que se comprometa con algún juicio basado en lo que hemos estudiado sobre lo que favorece, o desfavorece, la convergencia de diversos algoritmos).

Solución: Se tiene:

- 1) El método tiene la desventaja de utilizar información parcial del gradiente, que indica la dirección de máximo descenso. Una mala elección de la coordenada j_0 conllevaría a un descenso considerablemente peor que el método del gradiente, luego se espera que la cantidad de iteraciones que necesita sea considerablemente mayor. (Claramente esto implica que es mucho peor al compararlo con el Método de Newton en cantidad de iteraciones).
 - 2) Por otra parte, la complejidad del cálculo de la dirección d^k es, en general, mucho más barata que en el caso del Método del Gradiente (y por lo tanto que el Método de Newton). Esto implica que el algoritmo podría terminar antes que ambos métodos, en especial cuando N es muy grande.
- (e) Ahora se propone la modificación que viene en el siguiente pseudo-código, y a este segundo algoritmo lo llamaremos “Algoritmo Mas-Misterioso”:

Algoritmo Más-Misterioso (para minimizar):

Data: $x^0 \in \mathbb{R}^n$, f , $k = 0$, $LIMITE$, un entero > 0 , p , un entero entre 2 y n

while $k < LIMITE$ **do**

$y^1 = x^k$

for $i = 1, \dots, p - 1$ **do**

Seleccionar $j_0 \in \{1, \dots, n\}$ en forma aleatoria. Si j_0 es igual al de la selección anterior, repetir.

Sea $h_{j_0}^i = -\frac{\partial f}{\partial x_{j_0}}(y^i)$ Sea $h_j^i = 0, j = 1, \dots, n, j \neq j_0$

Resolver aproximadamente

$$\min_{\mu \geq 0} f(y^i + \mu h^i) \quad (\text{etapa de “linesearch”})$$

Sea μ^i solución de ese problema

Sea $y^{i+1} = y^i + \mu^i h^i$

$i = i + 1$

end

Sea $d^k = \frac{y^p - x^k}{\|y^p - x^k\|_2}$

Resolver aproximadamente

$$\min_{\lambda \geq 0} f(x^k + \lambda d^k) \quad (\text{etapa de “linesearch”})$$

Sea λ^k solución de ese problema

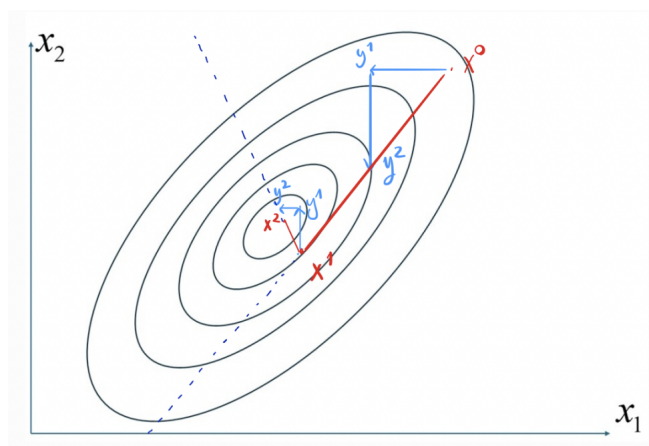
Sea $x^{k+1} = x^k + \lambda^k d^k$

$k = k + 1$

end

Explique qué hace esta modificación del algoritmo y haga un dibujo, similar al que hizo en la parte (b), mostrando la forma en que avanza este método. Explique qué ventaja considera usted que tiene este algoritmo comparado con la versión original (esto en el sentido de que pueda, por ejemplo, tener mejor convergencia). Justifique con precisión.

Solución: El algoritmo extiende el anterior, utilizando más coordenadas para “sondear” el espacio. Esto busca obtener información del entorno de la función para escoger de mejor manera la dirección de descenso, guiándose por donde hay mayor avance “potencial” de la función objetivo. Gráficamente:



Ejercicio 4

Nos interesa resolver el problema $\min_{x \in \mathbb{R}^n} f(x)$, donde $f : \mathbb{R}^n \rightarrow \mathbb{R}$ es dos veces continuamente diferenciable. Para encontrar una solución aproximada de este problema se propone el siguiente algoritmo:

Algoritmo 1:

```
Data:  $x^0 \in \mathbb{R}^n$ ,  $k = 0$ .  
while Criterio de parada no se satisface do  
    Sea  $\alpha = \max_{l=1,\dots,n} \left\{ \left| \frac{\partial f}{\partial x_l}(x^k) \right| \right\}$     if  $\alpha = 0$  then  
        | STOP  
    end  
    Sea  $j$  tal que  $\left| \frac{\partial f}{\partial x_j}(x^k) \right| = \alpha$  (si hay empate, se elije cualquiera)  
    if  $\frac{\partial f}{\partial x_j}(x^k) < 0$  then  
        |  $d^k = e^j$   
    else  
        |  $d^k = -e^j$   
    end  
    Resolver aproximadamente (hasta cumplir condición de Wolf-Armijo):  
  
        
$$\min_{t \geq 0} f(x^k + td^k)$$
  
  
        Sea  $t^k$  la solución elegida del problema anterior  
        Sea  $x^{k+1} = x^k + t^k d^k$   
         $k = k + 1$   
end
```

En este pseudo-código, e^j denota el j -ésimo vector canónico en $\mathbb{R}^n$ (el vector de n componentes, todas nulas, excepto la j -ésima que vale 1). No especificamos explícitamente el criterio de parada y no se preocupe de eso. Si quiere, piense en que es número de iteraciones.

- (a) Describa qué es lo que hace este algoritmo y por qué le parece que debería, efectivamente, servir para encontrar un mínimo local de la función. Su respuesta debe basarse en los contenidos del curso y su interpretación del pseudo-código, puede apoyarse también en representaciones gráficas, pero todo muy justificado.

Solución: Este algoritmo escoge en cada iteración la mayor derivada parcial en valor absoluto de la función objetivo con respecto a x en una dimensión en específico y lo llama α .

Si α es igual a cero, significa que se está en un mínimo local y se para de iterar (es equivalente a decir que el gradiente es igual a cero).

En el caso de que α sea distinto de cero, la derivada parcial de $f(x^k)$ con respecto a x_j , con $j \in 1, \dots, n$, que maximiza a α puede ser negativa o positiva. Si esta es menor que cero, se escoge como dirección de descenso el vector canónico positivo con un uno en la posición j de este (todas las demás posiciones con un cero). Si $\frac{\partial f(x^k)}{\partial x_j}$ es mayor que cero, para que disminuya el valor de la función objetivo se debe utilizar el negativo del mismo vector canónico descrito anteriormente.

Al final de la iteración, se resuelve aproximadamente el problema de minimizar la función evaluada en la suma de la solución actual más el paso de descenso ponderado. Se escoge el ponderador t del problema anterior, se define la solución x^{k+1} y se vuelve a iterar.

Este método encontraría un mínimo local, ya que para cada punto en cada iteración, decide moverse en la dirección de la variable que más hace decrecer el valor de la función objetivo, y se detendría cuando se encuentre en un mínimo.

- (b) ¿Qué ventajas o desventajas considera usted que este algoritmo tiene frente al Método del gradiente, que hemos estudiado en clases? En particular, ¿puede estimarse el orden de “flops” que este método necesita por iteración? De ser así, de una estimación y no olvide justificar y explicar bien sus supuestos.

Solución: La principal ventaja se basa en un caso de “suerte”, ya que el mayor problema del método del gradiente es que su dirección de descenso es muy local. Este algoritmo propuesto navega por distintas coordenadas, es decir, en distintas direcciones según iteración, las cuales detecta que disminuyen rápidamente el valor de la función objetivo. Así, dadas unas condiciones favorables, este algoritmo podría ser más eficiente que el método del gradiente, si es que lograr capturar mejor la curvatura.

Por otra parte, una desventaja que tiene es que la dirección de descenso de este mismo algoritmo solo tiene elementos no nulos en una dimensión. El método del gradiente tiene la mejor dirección de descenso localmente. Así, la mayor parte del tiempo, el gradiente entregaría una menor cantidad de iteraciones hacia el mínimo (salvo que se de el caso “de suerte” anterior).

Al igual que el método del gradiente, requiere $\mathcal{O}(n)$ flops. La razón es que, aunque se utiliza solo un elemento del gradiente, en cada iteración se debe calcular alfa, valor que implica resolver n derivadas parciales y encontrar la más grande en valor absoluto.

- (c) Considere ahora la siguiente modificación que introduciremos al método. Suponiendo que vamos a partir del punto x^0 , consideramos la matriz Hessiana de f en x^0 , $H = \nabla^2 f(x^0)$. Supongamos que determinamos que H es simétrica y definida positiva. Por lo tanto, es diagonalizable. Sean $v^1, v^2, \dots, v^n$ los vectores propios que se obtienen de la diagonalización y vamos a asumir que se han normalizados, es decir, $\|v^j\|_2 = 1, j = 1, \dots, n$. Proponemos el siguiente algoritmo, modificado, que usa las direcciones v^j en vez de los vectores canónicos e^j :

Algoritmo 2:

Data: $x^0 \in \mathbb{R}^n$, $k = 0$.

while *Criterio de parada no se satisface* **do**

 Sea $\alpha = \max_{l=1,\dots,n} \{|\nabla f(x^k)^\top v^l|\}$ **if** $\alpha = 0$ **then**
 | STOP

end

 Sea j tal que $|\nabla f(x^k)^\top v^j| = \alpha$ (si hay empate, se elige cualquiera)

if $\nabla f(x^k)^\top v^j < 0$ **then**

 | $d^k = v^j$

else

 | $d^k = -v^j$

end

 Resolver aproximadamente (hasta cumplir condición de Wolf-Armijo):

$$\min_{t \geq 0} f(x^k + td^k)$$

 Sea t^k la solución elegida del problema anterior

 Sea $x^{k+1} = x^k + t^k d^k$

$k = k + 1$

end

Explique por qué este algoritmo podría ser mejor que el algoritmo 1 presentado antes, en términos de número de iteraciones para llegar a una meta de error. ¿Qué desventajas podría tener, sin embargo, frente el método original? ¿Qué condición debería cumplir x^0 para que esta idea funcione bien, y por qué? Nuevamente, debe justificar en función de los contenidos del curso.

Solución: Este algoritmo tiene potencial de ser mejor que el anterior debido a que la dirección de descenso contiene información del Hessiano de la función, es decir, información de segundo orden. Podría decirse que sigue un “estilo Newtoniano”. La información de segundo orden que podemos extraer de este algoritmos involucra a los vectores propios, los cuales son utilizados como dirección de descenso. Estos determinan los ejes coordinados en que se mueven las direcciones de descenso, los que corresponderían a los ejes principales de la elipse de las curvas de nivel de la función objetivo. De esta manera, el algoritmo podría adaptarse a las características de la función en específico y eventualmente generar una menor cantidad de iteraciones.

Entre las desventajas que tiene este algoritmo, se encuentran el elevado costo computacional, y por consecuencia, un mayor tiempo de ejecución con respecto al primer algoritmo visto. Esto debido principalmente a que para comenzar a iterar, se deben hacer cálculos de gran magnitud, ya que se debe diagonalizar la matriz Hessiana para calcular sus vectores propios. Es importante notar que este cálculo se realiza una vez para todo el

uso del algoritmo. Además, cada iteración es un poco más costosa debido a que *alpha* implica calcular el gradiente de la función en x^k y multiplicarlo por cada vector propio calculado.

De esta manera, si la función no es bien comportada o posee un gran número de variables, puede que el método inicial fuese una mejor opción. Debe considerarse el balance entre número de iteraciones y el costo por iteración.

Por otro lado, sin el supuesto de que la función es estrictamente convexa y bien definida, se debe escoger cuidadosamente el punto de partida. Específicamente, suficientemente cerca del óptimo, donde sea localmente convexa la función objetivo. De no ser así, puede ocurrir que la sucesión contruida sea divergente o caiga en soluciones locales.

Ejercicio 5

Vamos a considerar el siguiente problema de estimación estadística. Tenemos m observaciones de un cierto fenómeno, $b_1, \dots, b_m$ y queremos explicarlo en función de n factores, para los cuales también existen valores conocidos a_{ij} , que es el valor del factor j para la observación i . Entonces, se postula un modelo lineal de la forma:

$$\sum_{j=1}^n a_{ij}x_j = b_i, \quad i = 1, \dots, m$$

en donde los x_j son los pesos de los factores. Entonces, queremos encontrar la mejor solución posible al sistema de ecuaciones $Ax = b$. Este problema lo hemos planteado ya en el curso, usando mínimos cuadrados pero acá usaremos un criterio distinto, el error en valor absoluto. Adicionalmente, por propiedades del problema subyacente, se sabe que los coeficientes de la estimación, las variables x , deben estar dentro de ciertos rangos específicos y conocidos. Entonces, se plantea resolver el siguiente problema de optimización:

$$\begin{aligned} \text{mín} \quad & \|Ax - b\|_1 \\ \text{s. a.} \quad & l_j \leq x_j \leq u_j, j = 1, \dots, n \end{aligned}$$

donde $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$ y los l_j, u_j son todos escalares conocidos. Como usted debe observar, la función objetivo de este problema es no diferenciable.

- (a) Determine un subgradiente de la función objetivo del problema en un punto x factible. (Indicación: le recordamos una relación que se ha usado ya en el curso, para hacer “regla de la cadena no diferenciable”, sea f una función convexa, si $p \in \partial f(x)$ y $g(x) = f(Bx)$, donde B es alguna matriz de dimensiones adecuadas, entonces $B^\top p \in \partial g(x)$.)

Solución: Sea $f(x) = \|Ax - b\|_1 = \sum_{i=1}^m |\alpha_i^\top x - b_i|$

Luego, se deriva usando la regla de la cadena:

$$\partial f(x) = \sum_{i=1}^m \partial |\alpha_i^\top x - b_i| = \sum_{i=1}^m \alpha_i \cdot p_i(x)$$

Donde definimos $p(x) \in \mathbb{R}^m$ como

$$p_i(x) = \begin{cases} 1 & \text{si } \alpha_i^\top x - b_i > 0 \\ -1 & \text{si } \alpha_i^\top x - b_i < 0 \\ u \in [-1, 1] & \text{si } \alpha_i^\top x - b_i = 0 \end{cases}$$

Finalmente, denotamos $h(x) \in \partial f(x)$ tal que:

$$h(x) = \sum_{i=1}^m \alpha_i p_i(x)$$

Una forma alternativa es derivando utilizando la indicación del enunciado:

$$\partial f(x) = \partial \|Ax - b\|_1 = A^\top \partial \|Ax - b\|_1 = A^\top p(x)$$

Con $p(x)$ definido anteriormente.

- (b) Explique, en forma detallada y con el desarrollo matemático correspondiente, cómo manejar las restricciones $l_j \leq x_j \leq u_j, j = 1, \dots, n$ del problema, si se usa el método de subgradiente para el problema.

Solución: Para poder manejar las restricciones, es necesario utilizar el Método del Subgradiente Proyectado. Para esto, se debe proyectar x en cada iteración del algoritmo, sobre el conjunto $D = \{x : \forall j \ l_j \leq x_j \leq u_j\}$.

Lo que se debe hacer es revisar cada cordenada x_j y determinar si es que está se encuentra dentro del conjunto D. Si $l_j \leq x_j \leq u_j$, entonces esta coordenada queda igual. Por otro lado, si $x_j < l_j$, esta coordenada se debe poner en l_j y si $x_j > u_j$, se debe llevar a u_j . Esto es:

$$\Pi_D(x)_j = \begin{cases} x_j & \text{si } l_j \leq x_j \leq u_j \\ l_j & \text{si } x_j < l_j \\ u_j & \text{si } x_j > u_j \end{cases}$$

Se puede apoyar la respuesta con un dibujo.

- (c) Ahora le proponen hacer una transformación al problema y poder resolverlo con algún algoritmo, ojalá, más eficiente que el método de subgradiente. Para esto, considere la función $g : \mathbb{R} \rightarrow \mathbb{R}$ dada por $g(u) = \sqrt{u^2 + \mu}$ donde $\mu > 0$ es un parámetro dado. Muestre que $g(u) \approx |u|$, es decir, se puede usar para aproximar el valor absoluto, y que esa aproximación se hace cada vez mejor mientras más chico sea μ (puede dar un argumento simple, basado en aproximaciones matemáticas y apoyarse, también, en un gráfico, si lo desea).

Solución:

Se tiene que

$$g(u) = \sqrt{u^2 + \mu} = \sqrt{u^2(1 + \frac{\mu}{u^2})} = |u| \sqrt{1 + \frac{\mu}{u^2}}$$

Para valores de u muy grandes, $\frac{\mu}{u^2} \rightarrow 0$, por lo que $\sqrt{1 + \frac{\mu}{u^2}} \approx \sqrt{1} = 1$. Luego

$$g(u) \approx |u| \cdot 1 = |u|$$

Por otro lado, para valores de u más pequeños, la aproximación no sería tan buena. Ahora bien, si $\mu \rightarrow 0 \Rightarrow \frac{\mu}{u^2} \rightarrow 0$, por lo que a través del mismo razonamiento anterior se concluye que $g(u) \approx |u|$.

- (d) Dado lo anterior, se propone ahora resolver, alternativamente, el problema:

$$\begin{aligned} \text{mín} \quad & \sum_{i=1}^m g(\alpha_i^\top x - b_i) \\ \text{s. a.} \quad & l_j \leq x_j \leq u_j, j = 1, \dots, n \end{aligned}$$

donde $\alpha_1, \dots, \alpha_m$ son las filas de la matriz A , usando un algoritmo basado en el Método Acelerado de Nesterov (no es necesario recordar todo el detalle). ¿Puede usarse el Método Acelerado de Nesterov, con alguna etapa adicional, para este problema? Justifique con precisión sobre la base de las hipótesis necesarias para que funcione el algoritmo y otras propiedades que hemos visto en el curso, no tiene que describir el detalle del algoritmo.

Solución: La respuesta es sí. En primer lugar, es importante identificar que se debe usar el Método Acelerado de Nesterov Proyectado, ya que el problema incluye restricciones acotadas en x , lo que requiere proyectar las soluciones propuestas en cada iteración sobre el conjunto factible $[l, u]$. Esta etapa se incorpora en el algoritmo y se maneja de la misma manera que se menciona en el inciso b).

En segundo lugar, definimos

$$f(x) = \sum_{i=1}^m g(\alpha_i^\top x - b_i)$$

Luego, se observa que se cumplen los supuestos necesarios para poder utilizar este algoritmo. Estos son:

- Función convexa.

Esto se puede ver ya que la función $(\cdot)^2$ es convexa y la función $\sqrt{\cdot}$ es no decreciente. Se puede argumentar a través de un gráfico.

- Función doblemente diferenciable.

Como se aproxima la función valor absoluto mediante $g(u)$, la función ahora es diferenciable, ya que se suaviza su comportamiento para valores de u pequeños, mediante el parámetro μ .

- $\nabla f(x)$ L -Lipschitz.

Dado que la solución se busca en un conjunto acotado, luego se sabe que la norma del Hessiano alcanzará su máximo dentro de este, por lo que es posible acotarlo.